Universitat d'Alacant
Universidad de Alicante

Práctica 01

Sesión 3 y 4 - Implementación del algoritmo Forward Checking



Nombre: Daironas Sukis

NIE: Y1759578T

Grupo: 01

Asignatura: Sistemas Inteligentes, Ingeniería Informática, Universidad de Alicante

1. Contexto....3
2. Modificación del variable.py....3
3. Modificación del main.py....3
3.1 Función esConsistenteFC....3
3.2 Función forward....4
3.3 Función restaurar....5
3.4 Función FC....6
3.5 Reorganización de la función main....7
3.6 Guardar y cargar crucigramas....8
4. Ejecución del algoritmo Forward Checking....10
4.1 Tablero 3x3 con solución....10
4.2 Tablero 5x4 sin solución....11

1. Contexto

En esta sesión se pide implementar el algoritmo de *Forward Checking (FC)*, una mejora sobre el backtracking visto en la sesión anterior. Mientras que backtracking solo comprueba si una asignación es consistente con las variables ya asignadas, Forward Checking va un poco más allá, ya que cada vez que asigna un valor a una variable, se revisan los dominios de las variables futuras, y se eliminan temporalmente las palabras que no serían compatibles con dicha asignación. Así si conseguimos que el dominio de una variable futura se quede vacía, se descarta la asignación actual sin la necesidad de seguir explorando ramas que ya sabemos que van a fallar.

2. Modificación del variable.py

Cuando tengamos que podar una palabra del dominio de la variable, tendremos que eliminarla del dominio, pero si llegamos a necesitar deshacer esa poda, necesitamos un nuevo atributo en la clase, una lista que almacene la tupla **(palabra_podada, variable_responsable)**, en este caso lo hemos llamado **podado**.

```
class Variable:
    def __init__(self, nombre, tipo, fila, col, longitud, dominio):
        self.nombre = nombre
        self.tipo = tipo        # h/v
        self.fila = fila
        self.col = col
        self.longitud = longitud
        self.dominio = dominio
        self.restricciones = [] # lista de tuplas (vecino, pos_propia, pos_vecino)
        self.valor = None
        self.podado = []    # lista de tuplas (palabra_podada, variable_responsable)
```

Es necesario guardar que variable ha sido la responsable en podar, ya que de esta manera sabremos qué podas deshacer para cada variable concreta cuando esta se desasigne, sin afectar a las podas que hayan hecho otras variables que siguen asignadas en ese momento.

3. Modificación del main.py

3.1 Función esConsistenteFC

```python
def esConsistenteFC(var_i ,valor_i, var_j, valor_j):
    for restriction in var_i.restricciones:
        if restriction[0] == var_j:
            if valor_i[restriction[1]] == valor_j[restriction[2]]:
                return True
            else:
                return False
    return True
```

Esta función comprueba si dos valores concretos, de dos variables concretas, son compatibles entre sí en el punto que se cruzan. Recorremos las restricciones de *var_i*, buscando la que corresponde con *var_j*, y comprueba que el carácter donde se cruzan coincidan con el valor de cada una. Como dos variables solo pueden cruzarse como máximo una vez, por eso retornamos nada más realizar esa comprobación. Si no encuentra como restricción a la *var_j* buscada, significa que las variables no se cruzan, por lo que no pueden entrar en conflicto.

Es importante recalcar que esta función es distinta de *isFeasible*, ya que este comprueba si una palabra es válida para una variable teniendo en cuenta todas sus restricciones y que no esté repetida en el crucigrama. En cambio *esConsistenteFC* solo comprueba que son compatibles dos variables específicas, con dos valores concretos, usada por *forward* para decidir qué palabras podar.

3.2 Función forward

```python
def forward(k, listaVariables):
    dominio_vacio = False
    var_i = listaVariables[k]
    valor_i = var_i.valor

    for j in range(k+1, len(listaVariables)):
        var_j = listaVariables[j]

        for a in var_j.dominio.copy():
            if not esConsistenteFC(var_i, valor_i, var_j, a):
                var_j.dominio.remove(a)
                var_j.podado.append((a, var_i))
```

```
        if len(var_j.dominio) == 0:
            dominio_vacio = True
            break
    return not dominio_vacio
```

Una vez que se haya asignado un valor a la variable actual *var_i*, esta función recorre todas las variables futuras, desde *k+1 (la siguiente variable de la actual)*, hasta el final de la lista. Para cada una de las variables que vamos a recorrer, se va a recorrer su dominio, comprobando si cada palabra sigue siendo consistente con la asignación recién hecha. Recorremos la copia del dominio en el bucle, ya que estamos modificando el propio dominio y así evitamos problemas mientras se modifica.

Si en la comprobación encontramos que una palabra deja de ser consistente, se elimina del dominio de esa variable futura y se guarda en la lista de podado una tupla *(palabra_podada, variable_responsable)*, para poder deshacer esta poda más adelante si hace falta.

Después de haber revisado el dominio completo de una variable futura, se comprueba si se ha quedado vacío, si es así se marca *dominio_vacio* = *True*, y cortamos el bucle ya que no tiene sentido seguir comprobando las demás variables, ya que no va a llevar a ninguna solución.

Al final devolvemos ***not dominio_vacio***, si se ha vaciado algún dominio devolverá *False*, si no devolverá *True*.

3.3 Función restaurar

```
def restaurar(k, listaVariables):
    var_i = listaVariables[k]

    for j in range(k+1, len(listaVariables)):
        var_j = listaVariables[j]

        for elem_podado in var_j.podado.copy():
            palabra, responsable = elem_podado
            if responsable == var_i:
                var_j.podado.remove(elem_podado)
                var_j.dominio.add(palabra)
```

Esta función deshace lo que hizo forward para una asignación concreta. Recorre las mismas variables futuras *(desde k+1 hasta el final de la lista)*, y para cada una de estas

revisa la lista de podados, buscando las tuplas cuyo responsable haya sido la variable *var_i* que se está desasignando.

Si encuentra alguna, elimina la tupla de la lista de podados, y devuelve la palabra al dominio.

3.4 Función FC

```
def FC(k, listaVariables):
    if k == len(listaVariables):
        return True
    actualVar = listaVariables[k]
    for a in listaVariables[k].dominio.copy():
        if isFeasible(actualVar, a, listaVariables):
            actualVar.valor = a
            if forward(k,listaVariables):
                if FC(k+1, listaVariables):
                    return True
            actualVar.valor = None
            restaurar(k, listaVariables)
    return False
```

La estructura de esta función es muy parecida a BackTracking, pero con dos diferencias principales.

1. Antes de asignar un valor, se sigue usando *isFeasible*, ya que *Forward Checking* solo poda por cruces, mediante *forward*, pero no comprueba que la palabra no esté ya siendo usada en otra variable. Por lo que reutilizando *isFeasible* garantizamos que se cumpla también dicha regla.
2. Después de asignar la variable, llamamos a forward para podar las variables futuras. Solo si *forward* tiene éxito,sin dejar ningún dominio de las variables vacío, se continuará con la llamada recursiva *FC(k+1, listaVariables).*

Tanto si forward falla como si la llamada recursiva no encuentra solución, se desasigna la variable actual (*actualVar.valor = None*), y se llama a restaurar para devolver el dominio a esas variables futuras que se habían podado por culpa de esta asignación.

3.5 Reorganización de la función main

Con la incorporación de Forward Checking, el bloque de eventos del main empezaba a ocupar bastante código, y encima código repetitivo entre los botones *BK y FC*, ya que ambos siguen exactamente el mismo flujo: construir las variables, calcular las restricciones, ejecutar un algoritmo de búsqueda, y pintar la solución o avisar si no la hay. La única diferencia entre ambos botones es qué algoritmo se ejecuta. Para evitar esta repetición se han creado las siguientes dos funciones.

```
def obtenerVariables(tablero, almacen):
    variablesHorizontales, contador = construyeVariablesHorizontal(tablero,
almacen)
    variablesVerticales = construyeVariablesVertical(tablero, almacen,
contador)
    todasVariables = variablesHorizontales + variablesVerticales
    encuentraRestricciones(todasVariables)
    return todasVariables
```

```
def resuelve(algoritmo, tablero, todasVariables):
    if algoritmo(0, todasVariables):
        pintarSolucion(todasVariables, tablero)
    else:
        MessageBox.showwarning("Alerta", "No hay solución")
```

De esta manera en *obtenerVariables* agrupamos la construcción de las variables y el cálculo de las restricciones de dichas variables, mientras que en *resuelve* agrupamos la ejecución del algoritmo, el pintado de la solución, y el mensaje de error.

La función resuelve puede recibir backTracking o FC como parámetro del algoritmo ya que ambas funciones comparten la misma forma de parámetro de entrada, y devuelven True o False.

Además, en el main se ha añadido la variable *todasVariables*, que permite conservar las variables ya construidas si primero se pulsa el botón VAR y, a continuación, se resuelve el crucigrama con *BK* o *FC,* evitando así construirlas dos veces. De esta manera, evitamos trabajar con variables desactualizadas.

La variable *todasVariables* se pone a *None* cada vez que el estado del tablero cambia, es decir, si se edita una celda o cargamos un crucigrama distinto, para que la siguiente vez que se pulse un botón que necesite las variables, estas se reconstruyan a partir del tablero actual.

Tras ejecutar *BK* o *FC*, *todasVariables* también se pone a None, ya que las variables quedan en un estado ya resuelto y no tendría sentido reutilizarlas para una nueva resolución.

```
                    if todasVariables is None:
                        todasVariables = obtenerVariables(tablero, almacen)
                    resuelve(backTracking, tablero, todasVariables)
                    todasVariables = None

                elif pulsaBoton(pos, botFC):
                    if todasVariables is None:
                        todasVariables = obtenerVariables(tablero, almacen)
                    resuelve(FC, tablero, todasVariables)
                    todasVariables = None
```

3.6 Guardar y cargar crucigramas

Como una herramienta extra, se han añadido dos botones nuevos, *SAVE* y *LOAD*, que permiten guardar el estado actual del tablero en un fichero de texto, y volver a cargarlo más adelante, sin tener que redibujar el crucigrama cada vez que se quiera probar.

Guardar

```
def guardarCrucigrama(tablero, file):
    f = open(file, 'w', encoding="utf-8")

    for fila in range(tablero.getAlto()):
        celdas = []
        for col in range(tablero.getAncho()):
            celda = tablero.getCelda(fila, col)
            if celda == VACIA:
                celdas.append('0')
            elif celda == LLENA:
                celdas.append('1')
            else:
                celdas.append(celda)
```

```
        f.write(' '.join(celdas) + '\n')
    f.close()
```

A resumidas cuentas, esta función recorre el tablero celda por celda escribiendo un 0 para las celdas vacías, un 1 para las celdas negras, y el propio carácter para las celdas con una letra fija. Cada fila del fichero representa una fila del tablero, con los valores separados por espacios.

Al pulsar el botón *SAVE*,se pide al usuario un nombre del fichero sin la extensión .txt.

```
elif pulsaBoton(pos, botGuardarCrucigrama):
                    nombre = askstring("Guardar", "Nombre del fichero (sin .txt)")
                    if nombre:
                        guardarCrucigrama(tablero, nombre + ".txt")
                    print("Crucigrama guardado")
```

Cargar

Con el objetivo de leer estos ficheros, se ha añadido un método a la clase Tablero:

```
    def cargar(self, fichero):
        f = open(fichero, "r", encoding="utf-8")
        self.tablero = []

        fila = -1
        for linea in f:
            celdas = linea.split()
            if len(celdas) > 0:
                fila += 1
                self.tablero.append([])
                for car in celdas:
                    if car == '0':
                        self.tablero[fila].append('-')
                    elif car == '1':
                        self.tablero[fila].append('*')
                    else:
                        self.tablero[fila].append(car.upper())
        f.close()

        self.ancho = len(self.tablero[0])
        self.alto = len(self.tablero)
```

Se encarga de leer el fichero de línea en línea para traducir cada uno de los valores a las celdas, ignorando las líneas vacías.

Al pulsar el botón *LOAD*, se abre un explorador de archivos para elegir el fichero:

```
elif pulsaBoton(pos, botCargarCrucigrama):
                    ruta = askopenfilename(filetypes=[("Texto", "*.txt")])
                    if ruta:
                        tablero.cargar(ruta)
                        if tablero.getAlto() != filas or tablero.getAncho() != cols:
                            MessageBox.showwarning("Alerta", "El crucigrama guardado no tiene el
tamaño del tablero actual")
                            tablero = Tablero(filas, cols)
                    todasVariables = None
```

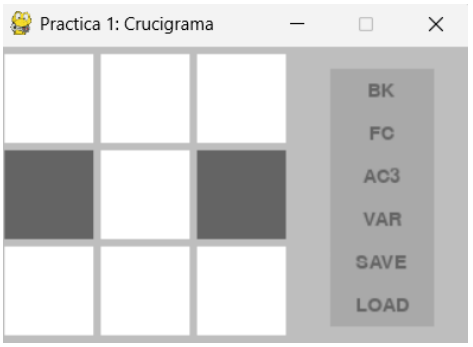
Solo se cargan crucigramas del mismo tamaño que el tablero actual, ya que aceptar un tamaño distinto obligaría a redimensionar la ventana y recalcular la posición de los botones, además de causar errores al intentar acceder a celdas fuera de rango.

4. Ejecución del algoritmo Forward Checking

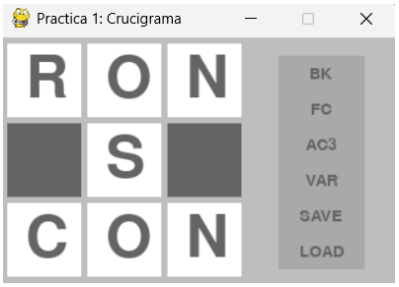
4.1 Tablero 3x3 con solución

Con un tablero de 3x3 y el fichero d0.txt, tapando las celdas (1,0) y (1,2):





Ejecutando el algoritmo de Forward Checking tenemos el siguiente resultado.



4.2 Tablero 5x4 sin solución

Con un tablero de 5x4 y el fichero d0.txt, tapando las celdas (0,0), (1,0), (2,0) y (3,3), e introduciendo el carácter L en la posición (3,1):

